

Architecture Decision Record

ADR-002

RabbitMQ para processamento assíncrono de pagamentos

Status: Aceite

Data	2026-05-08
Decisor	Equipa de Arquitectura
Projecto	Payment Service — Moçambique
Versão	1.0

Contexto

O processamento de pagamentos em Moçambique envolve chamadas a provedores externos (MPesa, e-Mola) que podem ter latência elevada ou falhar intermitentemente devido à baixa conectividade.

Processar de forma síncrona na thread HTTP bloquearia o cliente durante toda a duração da chamada ao provedor e reduziria drasticamente a resiliência e capacidade de resposta do sistema.

❑ *Requisito crítico: uma falha ou lentidão do provedor externo não deve ser visível para o cliente como um timeout — o sistema deve aceitar o pagamento imediatamente e processar em segundo plano.*

Decisão

Usar **RabbitMQ** com um Topic Exchange (`payment.exchange`) e três filas dedicadas:

Fila	Propósito
<code>payment.processing</code>	Envio ao provedor externo (MPesa / e-Mola)
<code>payment.notifications</code>	Notificações pós-processamento ao cliente
<code>payment.dead-letter</code>	Mensagens que esgotaram retries — análise e reproprocessamento

Razões

- **Desacoplamento** — o endpoint POST /payments responde imediatamente com 201 PENDING; o processamento pesado ocorre fora da thread HTTP.
- **Resiliência** — mensagens são persistidas em disco (durable: true); se a aplicação reiniciar, nenhuma mensagem é perdida.
- **Dead-letter queue** — pagamentos que falham após 3 retries (com backoff exponencial via @Retryable) caem na DLQ para análise e reproprocessamento manual ou automatizado.
- **Spring AMQP** — integração nativa com Spring Boot, incluindo Jackson2JsonMessageConverter para serialização JSON transparente.
- **Operacional simplificado** — RabbitMQ é mais simples de operar que Kafka em clusters pequenos; o Management UI facilita monitorização.

Alternativas Consideradas

Alternativa	Razão de Rejeição
Kafka	Overhead operacional elevado para o volume inicial; requer ZooKeeper ou KRaft; mais adequado para event streaming em larga escala
Processamento síncrono	Bloqueia a thread HTTP; falhas do provedor causam timeout directo para o cliente
SQS (AWS)	Vendor lock-in; não adequado para deploy on-premise em VPS

Consequências

A ter em conta para clientes

O estado de um pagamento pode ser PENDING por um curto período após o POST. Os clientes devem:

- Usar GET /payments/{id} para polling do estado final
- Ou implementar webhooks para receber notificações em tempo real

Operacionais

- A fila payment.dead-letter requer monitorização activa — configurar alertas quando mensagens entram nessa fila
- Em alta concorrência, o número de consumers pode ser escalado horizontalmente sem alteração ao código (basta adicionar réplicas do pod)

□ *Escalabilidade horizontal: aumentar o número de pods do payment-service aumenta automaticamente os consumers RabbitMQ sem qualquer alteração de configuração.*

Notas de Implementação

Configuração relevante do RabbitMQ (Spring AMQP):

```
// RabbitMQConfig.java
@Bean TopicExchange exchange() {
    return new TopicExchange("payment.exchange", true, false);
}

@Bean Queue processingQueue() {
    return QueueBuilder.durable("payment.processing")
        .withArgument("x-dead-letter-exchange", "payment.exchange")
        .withArgument("x-dead-letter-routing-key", "dead-letter")
        .build();
}

@Bean Queue deadLetterQueue() {
    return QueueBuilder.durable("payment.dead-letter").build();
}
```

Referências

- ADR-001 — PostgreSQL como base de dados principal
- ADR-003 — Idempotência via Idempotency-Key